



Practica 6:

CLASIFICADOR POR EL MÉTODO DEL PERCEPTRÓN SIMPLE EN PYTHON

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: **6CM3**

Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Notebook: [Practica no.6](#)

domingo, 16 de noviembre de 2025

CONTENIDO

MARCO TEÓRICO	3
PROCEDIMIENTO	6
DATOS DE ENTRENAMIENTO.....	6
<i>Grafica de dispersión</i>	6
FUNCIÓN DE ACTIVACIÓN ESCALÓN	7
FUNCIÓN DE ACTIVACIÓN	7
ENTRENAMIENTO DEL PERCEPTRÓN	8
<i>1er Entrenamiento</i>	8
<i>2do Entrenamiento</i>	10
<i>3er Entrenamiento</i>	11
<i>4to Entrenamiento</i>	12
<i>5to Entrenamiento</i>	13
<i>6to Entrenamiento</i>	14
<i>7mo Entrenamiento</i>	15
CONCLUSIONES	16
AGUIRRE LANTO VÍCTOR MANUEL.....	16
GASCA FRAGOSO PEDRO	16
GUEVARA BADILLO ARELI ALEJANDRA	16
MONTIEL TORO ARAEL DE JESÚS.....	17
RAMÍREZ LOZANO GAELE MARTIN.....	17

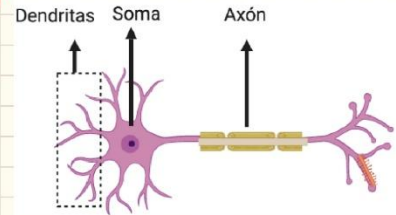
MARCO TEÓRICO

Composición de una neurona biológica y su funcionamiento

Una neurona biológica es la célula fundamental del sistema nervioso. Es la encargada de recibir, procesar y transmitir información mediante señales eléctricas y químicas.

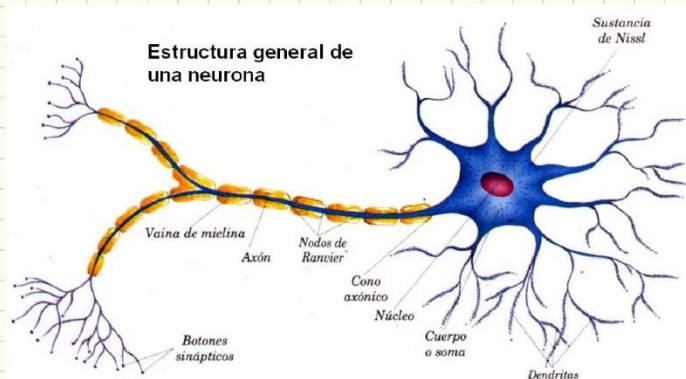
Composición

- ★ **Dendritas:** Son ramificaciones cortas que actúan como "antenas". Su función principal es recibir las señales (impulsos neuroeléctricos) de otras neuronas.
- Soma (Cuerpo Celular):** Es el centro de la neurona. Contiene el núcleo, que alberga el material genético. En el soma se integran o suman todas las señales recibidas por las dendritas.
- ★ **Axón:** Es una prolongación larga y única que transmite la señal de salida desde el soma hacia otras neuronas.
- ★ **Sinapsis:** Es el punto de conexión (aunque no hay contacto físico directo) entre el axón de una neurona y las dendritas de otra. Aquí es donde la señal "salta" de una célula a otra, usualmente mediante neurotransmisores.



Funcionamiento (Proceso de disparo):

- 1- **Recepción:** Las dendritas reciben miles de señales de entrada de otras neuronas.
- 2- **Integración:** El soma suma todas estas señales algunas pueden ser excitatorias, que "animan" a disparar, y otras inhibitorias, que "desaniman".
- 3- **Disparo Potencial de Acción:** Si la suma total de las señales supera un cierto umbral de activación, la neurona "se dispara".
- 4- **Transmisión:** Al dispararse, la neurona envía un impulso eléctrico único un potencial de acción a lo largo de su axón.
- 5- **Comunicación:** Este impulso llega a la sinapsis y libera neurotransmisores, que son recibidos por las dendritas de la siguiente neurona, repitiendo el ciclo.



Modelo matemático de una neurona biológica para llegar al perceptrón

El perceptrón, desarrollado por Frank Rosenblatt, es el modelo matemático más simple de una neurona biológica. Es una "abstracción" que imita el proceso de decisión de "disparar o no disparar".

Para llegar a él, traducimos cada componente biológico a un término matemático:

- 1- Dendritas (Entradas): Se modelan como un vector de entradas, x . Cada entrada individual ($x_1, x_2, \dots, x_n$) representa una señal.
- 2- Sinapsis (Fuerza de conexión): Se modelan como pesos sinápticos, w . Cada entrada x_i tiene un peso asociado w_i . Este peso determina la importancia de esa entrada. Un peso alto significa que la entrada x_i es muy relevante para la decisión final.
- 3- Soma (Núcleo integrador): El "núcleo" del perceptrón realiza dos tareas que ocurren en el soma biológico:
 - ★ Combinación Lineal: Primero, suma todas las entradas multiplicadas por sus pesos ($x_i * w_i$). Esto imita la integración de señales.
 - ★ Sesgo Bias: Se añade un término llamado sesgo (b o w_0). Este término representa el "umbral de activación" base de la neurona. Es una medida de qué tan "fácil" o "difícil" es que la neurona se dispare, independientemente de las entradas.

La fórmula de esta combinación lineal que llamaremos z es:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

- 4- Axón (Salida): La señal de salida, y , se produce después de que z pasa por la Función de Activación. En el perceptrón simple, esta es una función escalón (o de Heaviside).

La función de activación imita el "disparo" de la neurona. Compara la suma ponderada (z) con un umbral (que gracias al sesgo, suele ser 0).

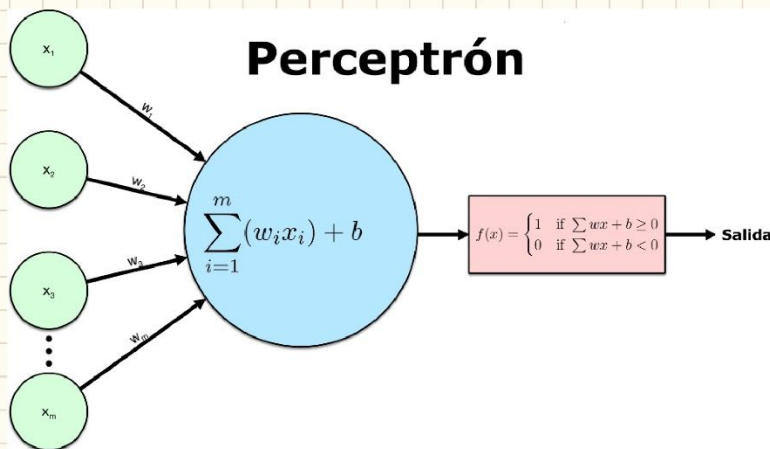
Si $z > 0$, la neurona "se dispara" $\rightarrow$ Salida = 1 Aprobada.

Si $z \leq 0$, la neurona "no se dispara" $\rightarrow$ Salida = 0 Denegada.

$$y = 1 \text{ si } z > 0 \quad y = 0 \text{ si } z \leq 0$$

Diagrama de estructura básica del perceptrón

El diagrama del perceptrón visualiza el flujo de información descrito en el modelo matemático. Muestra las entradas (x_i) a la izquierda, que se conectan al "núcleo" (el nodo central) a través de sus pesos (w_i). El núcleo contiene la función de suma (Sigma) y la función de activación (representada por f o ϕ), produciendo la única salida (y) a la derecha.

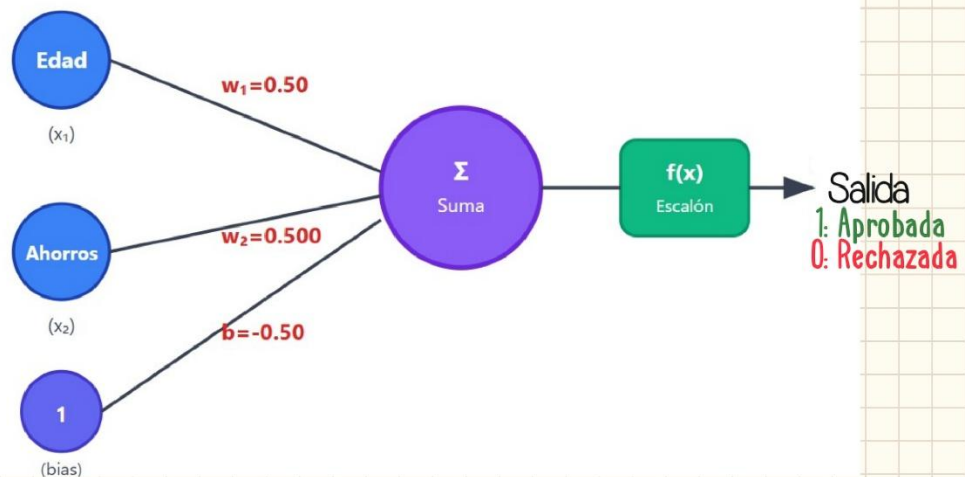


¿El perceptrón es un modelo de aprendizaje automático supervisado o no supervisado?

Sí, la razón por la cual el perceptrón es un modelo de aprendizaje supervisado es que aprende a partir de ejemplos que ya están "etiquetados" con la respuesta correcta, actuando estas etiquetas como un maestro o supervisor que guía el proceso. Durante el entrenamiento, el perceptrón realiza una predicción basada en las entradas y la compara inmediatamente con la etiqueta real, si su predicción es incorrecta, utiliza la diferencia del error para ajustar ligeramente sus pesos sinápticos y el sesgo. Este ciclo iterativo de corrección de errores es fundamental para su funcionamiento y solo es posible porque se le proporcionan de antemano las respuestas deseadas para que pueda calibrar su precisión.

Ecuación: $f(x) = \text{escalón}(w_1 \times \text{Edad} + w_2 \times \text{Ahorros} + \text{bias})$

Si $\Sigma \geq 0 \rightarrow \text{Salida} = 1$ (Aprobada) | Si $\Sigma < 0 \rightarrow \text{Salida} = 0$ (Denegada)



PROCEDIMIENTO

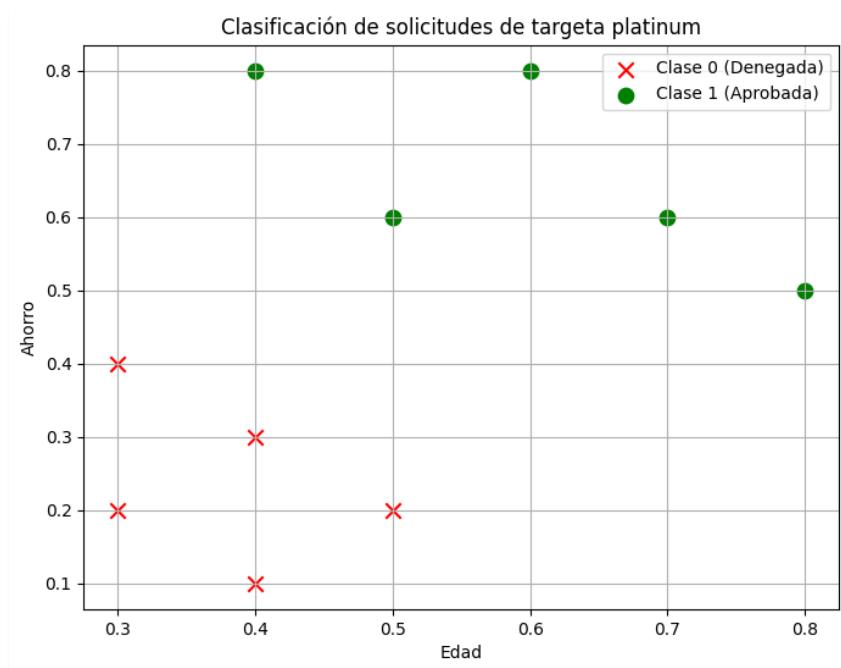
Datos de entrenamiento

```
1 # Datos de 10 personas -> [edad, ahorro]
2 # edad: 1 -> 100 años
3 # ahorros: 1 -> $100,000
4 personas = np.array([
5     [0.3, 0.4], [0.4, 0.3],
6     [0.3, 0.2], [0.4, 0.1],
7     [0.5, 0.2], [0.4, 0.8],
8     [0.6, 0.8], [0.5, 0.6],
9     [0.7, 0.6], [0.8, 0.5]
10 ])
11
12 #1: aprobada
13 #0: denegada
14 clases = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])
```

El primer arreglo *personas* se compone de vectores donde cada uno representa a un cliente, el primer valor representa su edad y el segundo los ahorros que tiene,

El segundo arreglo *clases* contiene la etiqueta correspondiente a cada cliente donde 1 es tarjeta aprobada y 0 tarjeta denegada.

GRAFICA DE DISPERSIÓN

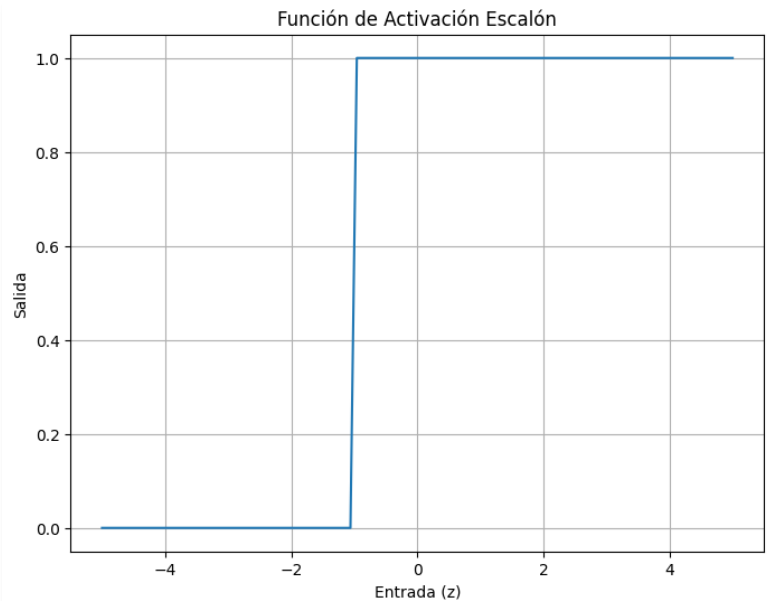


Función de Activación Escalón

```

1 def escalon(pesos, b, x):
2     z = pesos * x
3     if z.sum() + b > 0:
4         return 1
5     else:
6         return 0
7
8 pesos = np.random.uniform(-1, 1, size=2)
9 b = np.random.uniform(-1, 1)
10
11 # Generar valores para el eje x
12 x_values = np.linspace(-5, 5, 100)
13
14 # Aplicar la función escalón a los valores de x
15 y_values = [escalon(pesos, b, x) for x in x_values]
16
17 # Graficar la función escalón
18 plt.figure(figsize=(8, 6))
19 plt.plot(x_values, y_values)
20 plt.title('Función de Activación Escalón')
21 plt.xlabel('Entrada (z)')
22 plt.ylabel('Salida')
23 plt.grid(True)
24 plt.show()

```



El comportamiento de la función escalón es el de un **interruptor digital**: es una función "todo o nada". La neurona toma una decisión binaria basada en un umbral. Si la entrada neta z no supera ese umbral, la neurona permanece apagada con una salida de 0. En el instante en que la entrada z cruza el umbral y se vuelve positiva, la neurona se enciende bruscamente, y su salida salta instantáneamente a 1.

Función de Activación

```

1 # Definición de la función de activación
2 def activacion (pesos, x, b):
3     # Calcula la suma ponderada de las entradas y el sesgo
4     z = np.dot(pesos, x) + b
5     # Aplica la función escalón: retorna 1 si la suma es positiva, 0 en caso contrario
6     if z > 0:
7         return 1
8     else:
9         return 0

```

```

#Pruebas con nuevos clientes
nClientes = np.array([[0.3, 0.5], [0.2, 0.8]])
pesos = np.random.uniform(-1, 1, size=2)
b = np.random.uniform(-1, 1)
print("Pruebas con nuevos clientes:")
for cliente in nClientes:
    resultado_prediccion = activacion(pesos, cliente, b)
    print(f"Cliente: {cliente}, Predicción: {resultado_prediccion}")

*** Pruebas con nuevos clientes:
Cliente: [0.3 0.5], Predicción: 0
Cliente: [0.2 0.8], Predicción: 0

```

Entrenamiento del Perceptrón

```
1 # Función de entrenamiento
2 def entrenamiento(tasaAprendizaje, epochs, weights, bias):
3     # Ciclo de entrenamiento
4     print("Entrenamiento del perceptrón...")
5     for epoc in range(epochs):
6         errorTotal = 0
7         # Iterar sobre cada ejemplo de entrenamiento
8         for i in range(len(personas)):
9             # Calcular la predicción del perceptrón
10            prediccion = activacion(weights, personas[i], bias)
11            # Calcular el error
12            error = clases[i] - prediccion
13            errorTotal += error**2
14            # Actualizar pesos y sesgo
15            weights[0] += tasaAprendizaje * personas[i][0] * error
16            weights[1] += tasaAprendizaje * personas[i][1] * error
17            bias += tasaAprendizaje * error
```

```
1 # Guardamos los valores de incialización para los siguientes entrenamientos
2 pesos = np.random.uniform(-1, 1, size=2)
3 b = np.random.uniform(-1, 1)
```

1ER ENTRENAMIENTO

En este apartado se realiza el primer entrenamiento sugerido en la practica con los clientes [0.5, 0.5], [0.2, 0.6] y [0.6, 0.9].

Realizamos una copia de los valores de pesos y umbral para el entrenamiento, esto para poder seguir utilizando los mismos valores para los siguientes entrenamientos, ya que es necesario conservar los valores originales para comparar cada caso de prueba, ya que si se conservan las mismas variables en todos los entrenamientos estaría realizando un ajuste fino del primer entrenamiento y no serían más entrenamientos como tal.

```
1 # Copia de valores para primer entrenamiento
2 pesos_v1 = np.copy(pesos)
3 b_v1 = np.copy(b)
```

Valores antes del entrenamiento

```
# Nuevos clientes para probar
nuevos_clientes = np.array([[0.5, 0.5], [0.2, 0.6], [0.6, 0.9]])
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v1}")
print(f"Sesgo: {b_v1}")
for cliente in nuevos_clientes:
    print(f"\nCliente: {cliente}\n Resultado: {activacion(pesos_v1, cliente, b_v1)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.5 0.5]
Resultado: 1

Cliente: [0.2 0.6]
Resultado: 1

Cliente: [0.6 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.01, 100, pesos_v1, b_v1)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
# Realizar pruebas con los nuevos clientes y el perceptron entrenado
print(f"Pesos: {pesos_v1}")
print(f"Sesgo: {b_v1}")
for cliente in nuevos_clientes:
    print(f"\nCliente: {cliente}\n Resultado: {activacion(pesos_v1, cliente, b_v1)}")

Pesos: [0.40195911 0.08199627]
Sesgo: -0.22474285325950277

Cliente: [0.5 0.5]
Resultado: 1

Cliente: [0.2 0.6]
Resultado: 0

Cliente: [0.6 0.9]
Resultado: 1
```

2DO ENTRENAMIENTO

Entrenamiento realizado con tasa de aprendizaje 0.001 y el cliente [0.6, 0.9].

```
# Copia de valores el entrenamiento
pesos_v2 = np.copy(pesos)
b_v2 = np.copy(b)
```

Valores antes del entrenamiento

```
# Clientes para probar
pCliente = [0.6, 0.9]
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v2}")
print(f"Sesgo: {b_v2}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v2, pCliente, b_v2)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.6, 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.001, 100, pesos_v2, b_v2)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
print(f"Pesos: {pesos_v2}")
print(f"Sesgo: {b_v2}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v2, pCliente, b_v2)}")

Pesos: [ 0.52915911 -0.58460373]
Sesgo: 0.22325714674049735

Cliente: [0.6, 0.9]
Resultado: 1
```

3ER ENTRENAMIENTO

Entrenamiento realizado con tasa de aprendizaje 0.01 y el cliente [0.6, 0.9].

```
# Copia de valores para el entrenamiento
pesos_v3 = np.copy(pesos)
b_v3 = np.copy(b)
```

Valores antes del entrenamiento

```
# Clientes para probar
pCliente = [0.6, 0.9]
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v3}")
print(f"Sesgo: {b_v3}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v3, pCliente, b_v3)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.6, 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.01, 100, pesos_v3, b_v3)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
print(f"Pesos: {pesos_v3}")
print(f"Sesgo: {b_v3}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v3, pCliente, b_v3)}")

Pesos: [0.40195911 0.08199627]
Sesgo: -0.22474285325950277

Cliente: [0.6, 0.9]
Resultado: 1
```

4TO ENTRENAMIENTO

Entrenamiento realizado con tasa de aprendizaje 0.1 y el cliente [0.6, 0.9].

```
# Copia de valores para el entrenamiento
pesos_v4 = np.copy(pesos)
b_v4 = np.copy(b)
```

Valores antes del entrenamiento

```
# Clientes para probar
pCliente = [0.6, 0.9]
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v4}")
print(f"Sesgo: {b_v4}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v4, pCliente, b_v4)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.6, 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.1, 100, pesos_v4, b_v4)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
print(f"Pesos: {pesos_v4}")
print(f"Sesgo: {b_v4}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v4, pCliente, b_v4)}")

Pesos: [0.46595911 0.13899627]
Sesgo: -0.28474285325950216

Cliente: [0.6, 0.9]
Resultado: 1
```

Explicación de los resultados:

El efecto de la **tasa de aprendizaje** es controlar la magnitud con el que se actualizan los pesos y el sesgo.

Todos estos entrenamientos comenzaron con los mismos valores iniciales. Sin embargo, el resultado final fue diferente en cada uno, dependiendo únicamente de la tasa de aprendizaje.

Una tasa de aprendizaje más alta hace que el modelo intente corregir sus errores con "pasos" más grandes, modificando sus parámetros de forma más agresiva en cada época. Una tasa más baja lo hace con "pasos" más pequeños y cautelosos, por eso es necesario colocar una tasa equilibrada para no caer en errores de entrenamiento como el subajuste y el sobre ajuste.

5TO ENTRENAMIENTO

Entrenamiento realizado con epoch igual a 30 y el cliente [0.6, 0.9].

```
# Copia de valores para el entrenamiento
pesos_v5 = np.copy(pesos)
b_v5 = np.copy(b)
```

Valores antes del entrenamiento

```
# Clientes para probar
pCliente = [0.6, 0.9]
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v5}")
print(f"Sesgo: {b_v5}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v5, pCliente, b_v5)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.6, 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.01, 30, pesos_v5, b_v5)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
print(f"Pesos: {pesos_v5}")
print(f"Sesgo: {b_v5}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v5, pCliente, b_v5)}")

Pesos: [ 0.47395911 -0.35100373]
Sesgo: -0.06474285325950273

Cliente: [0.6, 0.9]
Resultado: 0
```

6TO ENTRENAMIENTO

Entrenamiento realizado con epoch igual a 120 y el cliente [0.6, 0.9].

```
# Copia de valores para el entrenamiento
pesos_v6 = np.copy(pesos)
b_v6 = np.copy(b)
```

Valores antes del entrenamiento

```
# Clientes para probar
pCliente = [0.6, 0.9]
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v6}")
print(f"Sesgo: {b_v6}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v6, pCliente, b_v6)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.6, 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.01, 120, pesos_v6, b_v6)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
print(f"Pesos: {pesos_v6}")
print(f"Sesgo: {b_v6}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v6, pCliente, b_v6)}")

Pesos: [0.40195911 0.08199627]
Sesgo: -0.22474285325950277

Cliente: [0.6, 0.9]
Resultado: 1
```

7MO ENTRENAMIENTO

Entrenamiento realizado con epoch igual a 300 y el cliente [0.6, 0.9].

```
# Copia de valores para el entrenamiento
pesos_v7 = np.copy(pesos)
b_v7 = np.copy(b)
```

Valores antes del entrenamiento

```
# Clientes para probar
pCliente = [0.6, 0.9]
# Realizar pruebas con los nuevos clientes
print(f"Pesos: {pesos_v7}")
print(f"Sesgo: {b_v7}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v7, pCliente, b_v7)}")

Pesos: [ 0.71595911 -0.47100373]
Sesgo: 0.7152571467404978

Cliente: [0.6, 0.9]
Resultado: 1
```

Entrenamiento del perceptrón

Entrenamos el perceptrón con los siguientes valores:

```
entrenamiento(0.01, 300, pesos_v7, b_v7)

Entrenamiento del perceptrón...
```

Valores después del entrenamiento

```
print(f"Pesos: {pesos_v7}")
print(f"Sesgo: {b_v7}")
print(f"\nCliente: {pCliente}\n Resultado: {activacion(pesos_v7, pCliente, b_v7)}")

Pesos: [0.40195911 0.08199627]
Sesgo: -0.22474285325950277

Cliente: [0.6, 0.9]
Resultado: 1
```

Explicación de los resultados:

Estos resultados demuestran un concepto fundamental del entrenamiento: **más épocas no siempre es mejor**.

El de 30 épocas encontró el "punto ideal" para ajustar los pesos y obtener la respuesta correcta, mientras que los de **120 y 300 épocas** demuestran que, al entrenar más, el modelo se "pasó" de ese estado ideal, quedando en un estado que clasifica incorrectamente a este cliente.

El objetivo es encontrar el punto óptimo donde el modelo ha aprendido los patrones generales sin haber memorizado el ruido, logrando así el mejor rendimiento posible.

CONCLUSIONES

Aguirre Lanto Víctor Manuel

Esta práctica fue fundamental para comprender la conexión directa entre la biología y la inteligencia artificial, demostrando cómo una estructura orgánica como la neurona puede modelarse matemáticamente para resolver problemas de clasificación binaria. La clave de este aprendizaje fue entender cómo las entradas, análogas a las dendritas biológicas, y sus respectivos pesos sinápticos se integran en un núcleo central mediante una operación de suma ponderada. Posteriormente, la aplicación de una función de activación de tipo escalón permite tomar una decisión binaria de disparo, imitando el potencial de acción de una neurona real. Este proceso simplificado ilustra poderosamente cómo sistemas complejos de decisión pueden construirse a partir de unidades fundamentales relativamente sencillas.

Gasca Fragoso Pedro

A través de los múltiples entrenamientos realizados, pude observar y analizar el impacto crítico que tiene la tasa de aprendizaje en el desempeño y convergencia del clasificador de perceptrón. Se hizo evidente que una tasa de aprendizaje demasiado pequeña resulta en un proceso de ajuste de pesos excesivamente lento, requiriendo muchas más épocas para alcanzar una solución aceptable. Por el contrario, una tasa muy grande puede provocar que el modelo oscile drásticamente alrededor del mínimo del error, dando pasos demasiado agresivos que le impiden estabilizarse en una configuración óptima de pesos. Queda claro que encontrar un valor equilibrado para este hiperparámetro es esencial para un entrenamiento eficiente y efectivo.

Guevara Badillo Areli Alejandra

Los experimentos llevados a cabo modificando el número de épocas de entrenamiento arrojaron una conclusión contraintuitiva pero vital: más iteraciones de entrenamiento no garantizan necesariamente mejores resultados de generalización. Los datos obtenidos mostraron casos donde el modelo lograba un punto ideal de clasificación con un número reducido de épocas, y que al continuar el entrenamiento innecesariamente, este comenzaba a degradar su desempeño en ciertos casos de prueba. Esto evidencia el riesgo de sobreajuste, donde el perceptrón memoriza el ruido o particularidades específicas de los datos de entrenamiento en lugar de aprender el patrón general subyacente, perdiendo capacidad para clasificar correctamente nuevos datos.

Montiel Toro Arael de Jesús

Con el desarrollo de esta práctica confirmamos que el perceptrón simple opera eficazmente como un modelo de aprendizaje supervisado, dependiendo enteramente de la disponibilidad de datos previamente etiquetados que actúan como un maestro durante el proceso. El mecanismo central de aprendizaje reside en la comparación constante entre la predicción generada por la neurona y la etiqueta real esperada. El cálculo de esta diferencia o error funciona como el motor matemático que dispara el reajuste iterativo tanto de los pesos sinápticos como del sesgo, permitiendo que la neurona refine progresivamente su frontera de decisión hasta lograr diferenciar correctamente las clases objetivo.

Ramírez Lozano Gael Martin

Si bien el perceptrón simple demostró ser una herramienta funcional y suficiente para separar linealmente los datos en el problema de aprobación de tarjetas de crédito planteado, esta práctica también sirvió para evidenciar sus limitaciones inherentes. Queda claro que este modelo básico fallaría ante problemas donde las clases no sean linealmente separables, es decir, donde no se pueda trazar una única línea recta para dividirlos. Este ejercicio establece así una base fundamental para comprender la necesidad de arquitecturas más complejas, como las redes neuronales multicapa, para abordar problemas de clasificación no lineales en escenarios del mundo real.